



Prepared by group 19

資料結構雜湊表測驗網站

Final project

CBF111023 李顥恩、CBF113055 戴永宸

指導老師：蕭文峰



June, 2026

系統目標

1.線上互動測驗平台

即時測驗
立即查看答案

2.建立雜湊表題庫系統

管理題目
新增/修改題目

3. 自動批改

顯示正確答案

4. 建立線上測驗功能

隨機出題
隨機選項

5. 提升學習效果

錯題練習
重複測驗



基本功能

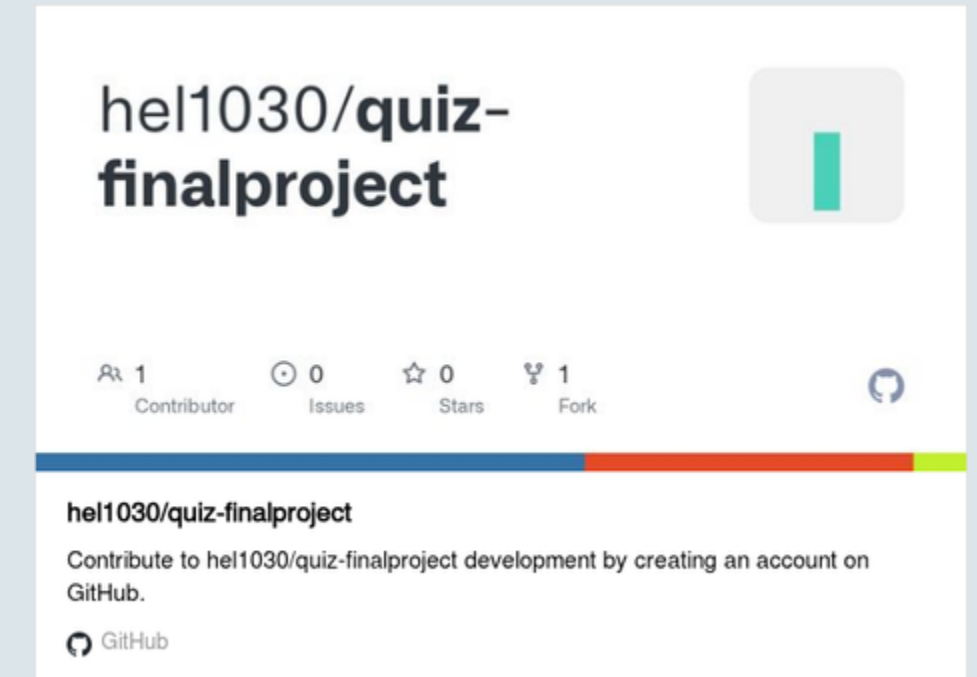
- 題庫後台管理
- 隨機出題
- 隨機選項排列
- 即時回饋
- 測驗題數 5 / 10 題
- 錯題重複出現

進階功能

- 登入與註冊
- 個人作答紀錄
- 排行榜
- 題目難易度

系統架構

專案使用 Django 製作資料結構「雜湊表」章節的線上測驗系統。系統包含登入、註冊、題庫後台管理、隨機出題、隨機選項排列、即時回饋、作答紀錄與排行榜。



使用者(User) → Django Web Server → 題庫系統(Database) → 測驗功能 → 成績顯示

github連結：

<https://github.com/hel1030/quiz-finalproject>

Explain

一、專題動機與目標

為什麼我們要做？

傳統紙本測驗：

- 題目固定
- 無法即時檢討
- 不易追蹤學習成果

專題目標

我們建立一套：

- 線上測驗系統
- 題庫管理系統
- 錯題複習系統



二、系統功能總覽

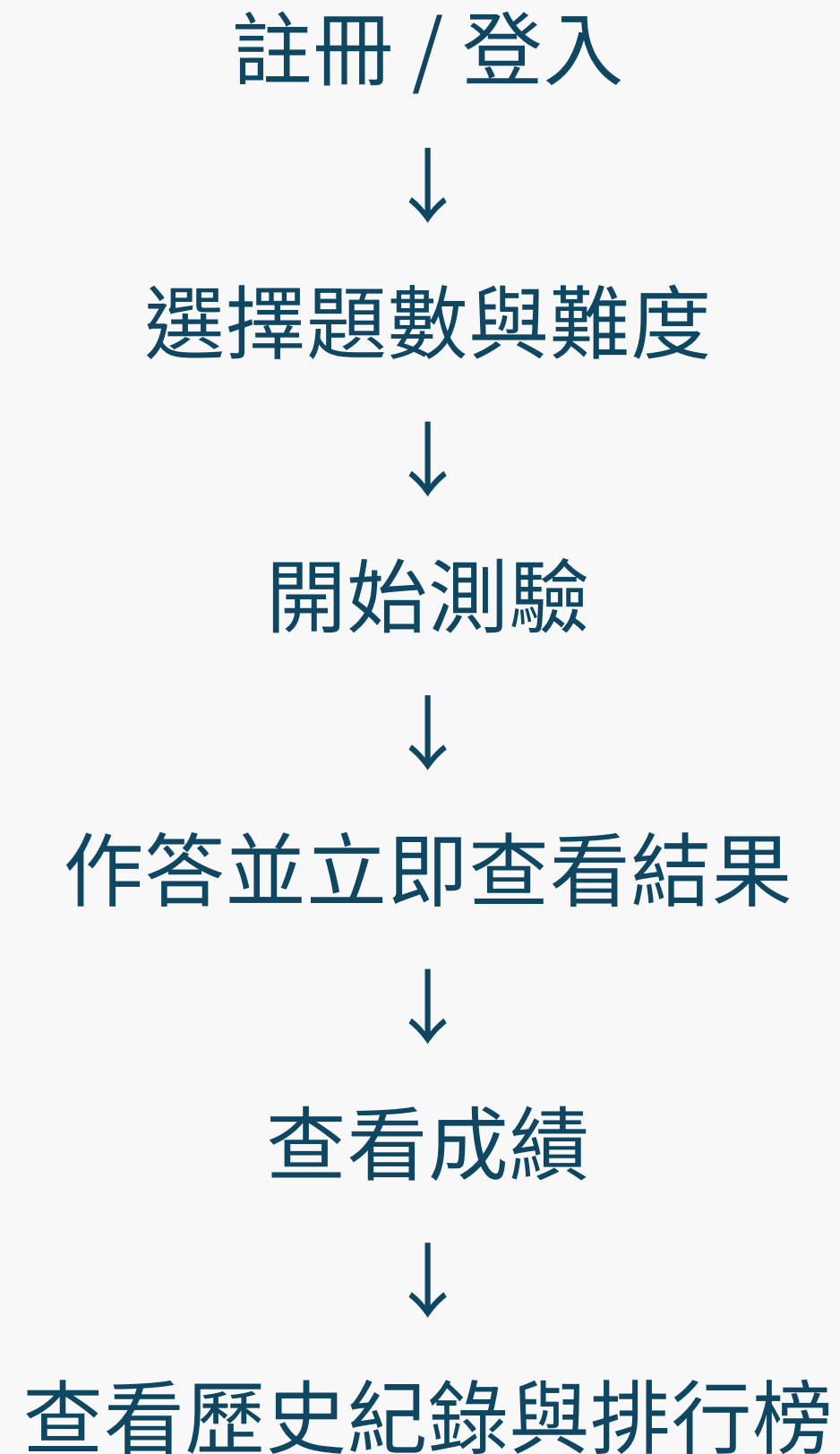
基本功能

- 題庫管理
- 隨機出題
- 隨機選項
- 即時回饋
- 題數選擇

進階功能

- 進階功能
- 登入註冊
- 作答紀錄
- 排行榜
- 錯題複習
- 難易度分級

三、使用者操作流程



流程圖

```
register / login | v home / setup ← 選題數、難度 | v (POST)
quiz_start ← 選題、建立 Session 和 QuizAttempt | v
quiz_question ← 顯示當前題目 (迴圈) | v (POST) quiz_answer ← 判
對錯、存資料庫、更新錯題紀錄 | (未做完) → 回到 quiz_question (做
完) ↓ quiz_summary ← 顯示結果總覽
```


QuizAnswer（每題作答明細）

```
class QuizAnswer(models.Model):  
    attempt = models.ForeignKey(QuizAttempt, ...) # 屬於哪次測驗  
    question = models.ForeignKey(Question, ...)  
    selected_option = models.ForeignKey(Option, ...) # 學生選的答案  
    correct_option = models.ForeignKey(Option, ...) # 正確答案  
    is_correct = models.BooleanField()
```

說明： 每做一題就寫一筆。紀錄學生答了什麼、正確答案是什麼，讓之後的歷史詳情頁可以顯示每題對錯。

UserAnswerHistory（錯題複習狀態）

```
python class UserAnswerHistory(models.Model):  
    user = models.ForeignKey(User, ...)  
    question = models.ForeignKey(Question, ...) is_correct = models.BooleanField() # 最近  
    一次是否答對 wrong_count = models.PositiveIntegerField() # 累計答錯次數  
    next_review = models.DateTimeField() # 下次複習時間
```

核心演算法 — 間隔重複 (Spaced Repetition) :

```
def update_spaced_repetition(self, answered_correct: bool):  
    if answered_correct: self.is_correct = True days = min(2 ** max(self.wrong_count - 1,  
0), 14) # 指數間隔，最長 14 天 else: self.is_correct = False self.wrong_count += 1 days  
= 1 # 答錯就明天再複習  
  
self.next_review = timezone.now() + timezone.timedelta(days=days)  
self.save()
```

說明：

- 答錯 → wrong_count + 1，明天（1 天後）再出現 - 答對 → 根據以前答錯幾次，決定多少天後再出現（ $2^{\{\text{錯誤次數}-1\}}$ 天，最多 14 天） - 例如：錯過 3 次後才答對 → $2^{\{3-1\}} = 4$ 天後再出現

四、系統架構

使用者 Browser



urls.py



views.py



models.py



SQLite Database

templates/

負責網頁顯示

補充：

- MVC/MVT 架構
- 前後端分離管理

五、資料表設計

主要資料表：

Question

Option

QuizAttempt

QuizAnswer

UserAnswerHistory

補充：

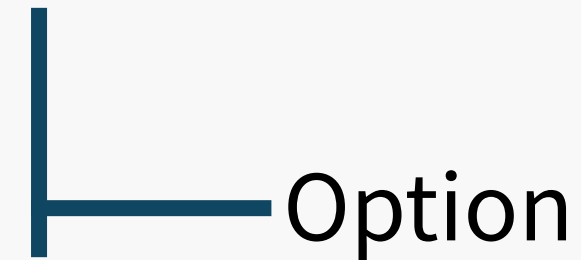
題目與選項分離

作答紀錄獨立保存

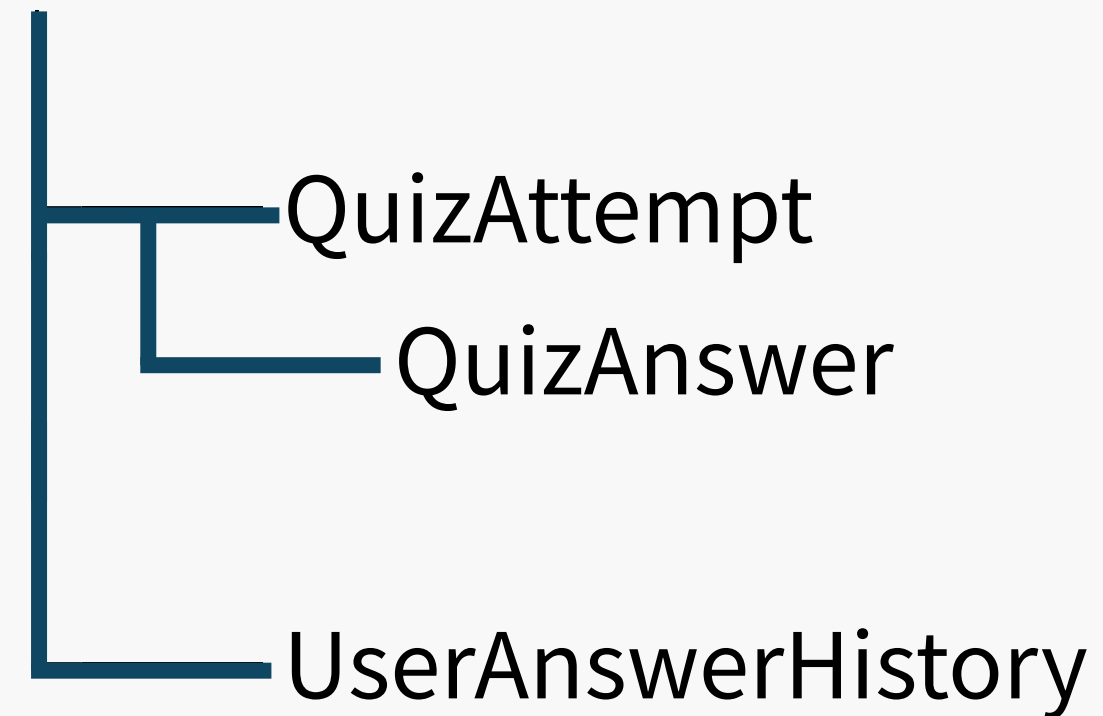
錯題歷史獨立管理

關聯圖：

Question



User



六、題庫管理機制

管理員可以：

新增題目

修改題目

刪除題目

管理選項

設定正確答案

(放 Admin 截圖)

優點：

不需額外開發後台

維護方便

七、隨機化機制

題目隨機

每次測驗重新抽題

選項隨機

每題選項重新排列

效果：

避免背答案位置

增加測驗公平性

流程：

取得題目



Shuffle 題目



取得選項



Shuffle 選項



存入 Session

八、即時回饋機制

回饋內容：

是否答對

使用者答案

正確答案

題目解析

特色：

立即學習、立即修正。

學生作答



取得選項 ID



判斷是否正確



顯示結果

九、錯題複習機制

資料表：

UserAnswerHistory

紀錄：

錯誤次數

最後作答時間

下次複習時間

特色：

類似 Spaced Repetition（間隔複習）

概念。

答錯題目



紀錄錯題



設定下次複習時間



下次測驗優先出題

十一、作答紀錄與排行榜

作答紀錄

測驗時間

分數

正確率

題目明細

排行榜

取每位使用者最高分

依成績排序

(放排行榜截圖)

效果：

增加學習動機與競爭性。

1. 取得難度篩選條件

```
difficulty = request.POST.get("difficulty", "")
```

2. 從題庫取出符合條件的題目

```
questions = Question.objects.filter(chapter=chapter,  
options__is_correct=True).distinct()
```

3. 錯題優先排序

```
due_ids = set(  
    UserAnswerHistory.objects.filter(user=request.user,  
is_correct=False) .filter(Q(next_review__isnull=True) |  
    Q(next_review__lte=timezone.now()))  
    .values_list("question_id", flat=True) )  
review_questions = [q  
for q in pool if q.id in due_ids] # 錯題  
new_questions = [q for  
q in pool if q.id not in due_ids] # 新題  
selected =  
(review_questions + new_questions)[:count] # 錯題排前面
```

4. 打亂選項順序並存入 Session（避免重新整理後選項改變）

```
option_order = {}  
for question in selected:  
    options = list(question.options.all())  
    random.shuffle(options)  
    option_order[str(question.id)] = [o.id for o in options]
```

5. 建立 QuizAttempt 紀錄

```
attempt = QuizAttempt.objects.create(...)
```

6. 全部存入 Session

```
request.session["quiz_ids"] = [...]  
request.session["quiz_index"] = 0  
request.session["quiz_score"] = 0  
request.session["quiz_option_order"] = option_order  
request.session["quiz_attempt_id"] = attempt.id ```
```

quiz_answer (判斷對錯)

```
```python selected_id = request.POST.get("option_id") #  
拿到學生選的選項 ID selected =
Option.objects.filter(id=selected_id,
question=question).first() is_correct = bool(selected and
selected.is_correct) # 判斷對錯
```

## 更新分數到 Session

```
if is_correct: request.session["quiz_score"] += 1
```

## 寫入 QuizAnswer (每題一筆)

```
QuizAnswer.objects.create(attempt=attempt, question=question,
...)
```

## 更新錯題複習狀態 (間隔重複演算法)

```
history.update_spaced_repetition(is_correct)
```

## 切到下一題更新分數到 Session

```
request.session["quiz_index"] += 1 ```
```

## leaderboard (排行榜)

```
```python
```

每位使用者只保留最高分那筆

```
best_by_user = {}  
for attempt in attempts:  
    if attempt.user_id not in best_by_user:  
        best_by_user[attempt.user_id] = attempt
```

排序：分數高 > 答對率高 > 使用者名稱字母序

```
board = sorted(best_by_user.values(),  
key=lambda a: (-a.score, -a.accuracy,  
a.user.username)) ```
```

十二、成果與未來發展

已完成

- ✓ 題庫管理
- ✓ 隨機測驗
- ✓ 即時回饋
- ✓ 錯題複習
- ✓ 作答紀錄
- ✓ 排行榜
- ✓ 難易度分級

未來改進

- 倒數計時器
- 錯題本專區
- 更多資料結構章節
- 統計分析圖表

間隔重複 (Spaced Repetition)

仿照艾賓浩斯遺忘曲線設計：

情況 下次出現時間

答錯 明天 (1 天後)

答對 (曾錯 1 次) $2^0 = 1$ 天後

答對 (曾錯 2 次) $2^1 = 2$ 天後

答對 (曾錯 3 次) $2^2 = 4$ 天後

答對 (曾錯 5 次) $2^4 = 16$ → 上限 14 天

這是科學化記憶方法的實作，讓使用者在最容易遺忘的時間點複習。

選項亂序固定

測驗開始時就把選項順序打亂並存入 Session，確保使用者重新整理頁面時選項不會改變，防止作弊或混淆。

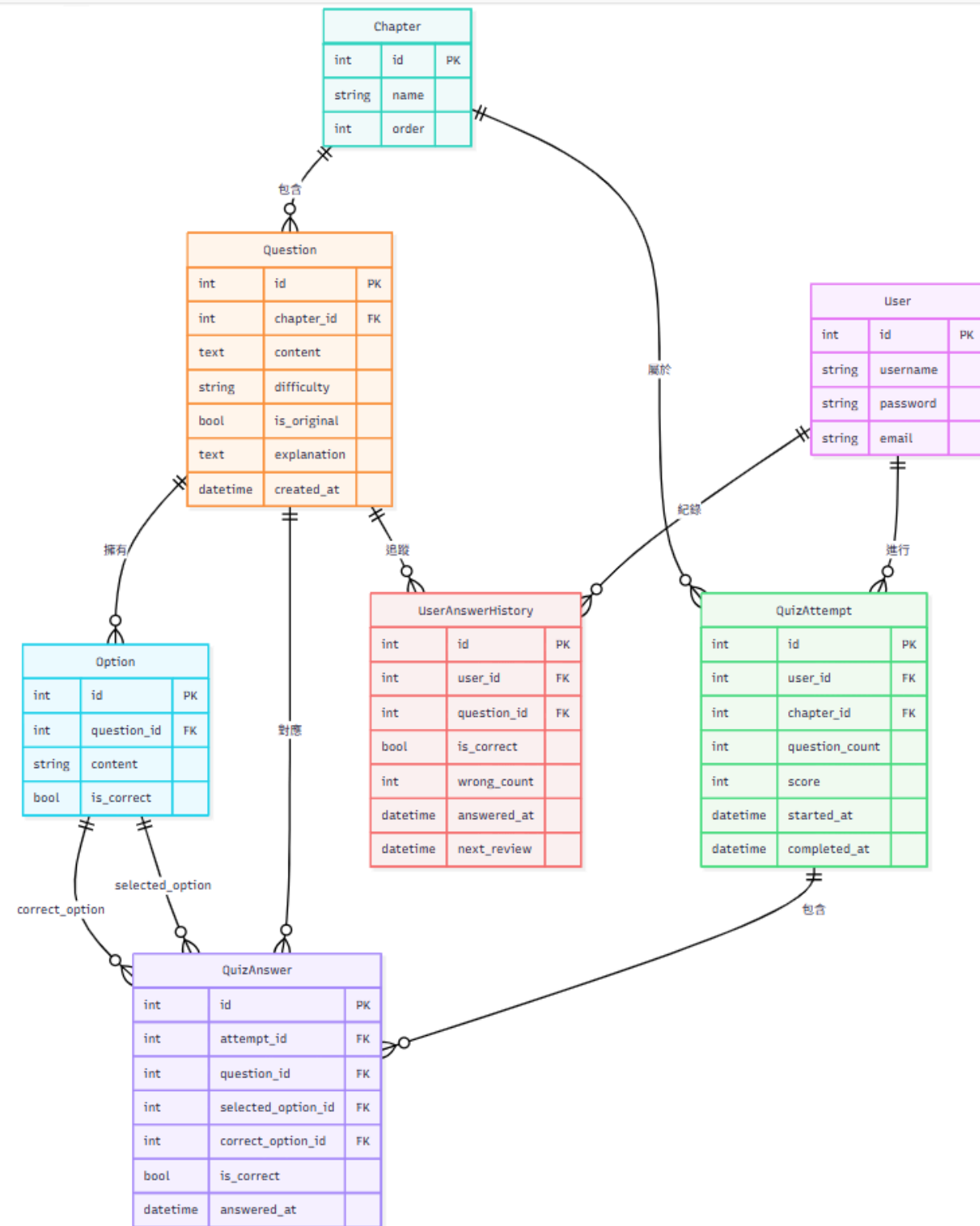
資料庫設計分離

QuizAttempt (整次紀錄) 和 QuizAnswer (每題明細) 分開儲存，讓歷史查詢與統計更清晰有效率，也方便未來擴充分析功能。

排行榜去重邏輯

每位使用者只取最高分紀錄，再以「分數 → 答對率 → 名稱字母序」三層排序，確保排名公平且不重複。

資料庫關聯圖



Github Project

執行網站

開啟瀏覽器進入：

<http://127.0.0.1:8000>

未登入時會先進入登入頁。
一般學生可透過註冊頁建立帳號。



啟動開發伺服器：

`python manage.py runserver`

系統需求

Python 3.13

Django 6.0.5

SQLite

使用 Conda 建立環境

教學團隊可在專案根目錄執行：

```
conda env create -f environment.yml
```

```
conda activate hash-quiz
```

確認 Django 版本：

```
python -m django --version
```

建立環境後，進入專案根目錄執行：

```
python manage.py migrate
```

若需要登入 Django Admin，請建立管理員帳號：

```
python manage.py createsuperuser
```

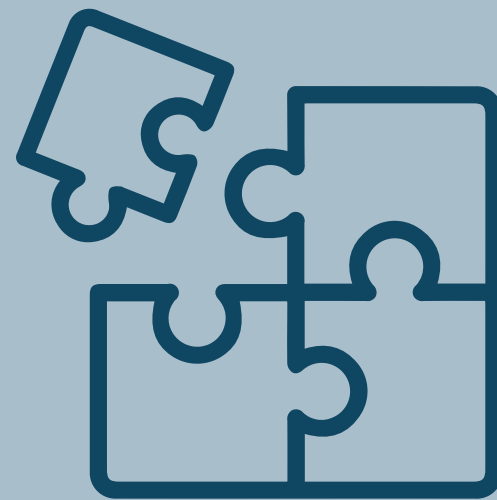
初始化資料庫

後台管理

<http://127.0.0.1:8000/admin/>



- 只有管理員帳號可以看到後台入口。後台可新增、修改、刪除題目與選項。



- 本專題章節固定為「雜湊表」，因此後台不提供新增章節功能。新增題目時系統會自動歸到雜湊表章節。



- 新增選項時只需要輸入選項文字，不需要輸入 A/B/C/D。測驗時系統會隨機排列選項，並依照當下順序顯示 A、B、C、D、E。

結論



本系統成功整合測驗、學習與複習功能，提升學生學習雜湊表的效率與互動性。

學習到：

Django網站開發

資料庫設計

前後端整合

同時也提升了：

資料結構理解

Web開發能力

團隊合作能力



Thank you

